P3395R1
Fix encoding issues and add a formatter for std::error_code

Published Proposal, 2025-03-12

Author:
Victor Zverovich

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

§ 1. Introduction

This paper proposes making `std::error_code` formattable using the formatting facility introduced in C++20 (`std::format`) and fixes encoding issues in the underlying API ([LWG4156]).

§ 2. Changes since R0

- Changed the title from "Formatting of std::error_code" to "Fix encoding issues and add a formatter for std::error_code" to reflect the fact that the paper also fixes [LWG4156].
- Specified that `error_category::name()` returns a string the ordinary literal encoding per SG16 feedback.
- Made transcoding in `error_category::message()` implementation-defined if the literal encoding is not UTF-8 per SG16 feedback and for consistency with other similar cases in the standard.

§ 3. Motivation

`error_code` has a rudimentary `ostream` inserter. For example:

```
std::error_code ec;
auto size = std::filesystem::file_size("nonexistent", ec);
std::cout << ec;
```

This works and prints `generic:2`.

However, the following code doesn't compile:

```
std::print("{}\n", ec);
```

Unfortunately, the existing inserter has several issues, such as I/O manipulators applying only to the category name rather than the entire error code, resulting in confusing output:

```
std::cout << std::left << std::setw(12) << ec;
```

This prints:

```
generic     :2
```

Additionally, it doesn't allow formatting the error message and introduces potential encoding issues, as the encoding of the category name is unspecified.

4. Proposal

This paper proposes adding a `formatter` specialization for `std::error_code` to address the problems discussed in the previous section.

The default format will produce the same output as the `ostream` inserter:

```
std::print("{}\n", ec);
```

Output:

```
generic:2
```

It will correctly handle width and alignment:

```
std::print("[{:>12}]\n", ec);
```

Output:

```
[   generic:2]
```

Additionally, it will allow formatting the error message:

```
std::print("{:s}\n", ec);
```

Output:

```
No such file or directory
```

(The actual message depends on the platform.)

The main challenge lies in the standard's lack of specification for the encodings of strings returned by `error_category::name` and `error_code::message` / `error_category::message` (syserr.errcat.virtuals):

```
virtual const char* name() const noexcept = 0;
```

Returns: A string naming the error category.

```
virtual string message(int ev) const = 0;
```

Returns: A string that describes the error condition denoted by `ev`.

In practice, implementations typically define category names as string literals, meaning they are in the ordinary literal encoding.

However, there is significant divergence in message encodings. libc++ and libstdc++ use `strerror[_r]` for the generic category which is in the C (not "C") locale encoding but disagree on the encoding for the system category: libstdc++ uses the Active Code Page (ACP) while libc++ again uses `strerror` / C locale on Windows. Microsoft STL uses a table of string literals in the ordinary literal encoding for the generic category and ACP for the system category.

The following table summarizes the differences:

	libstdc++	libc++	Microsoft STL
POSIX	`strerror`	`strerror`	N/A
Windows	`strerror` / ACP	`strerror`	ordinary literals / ACP

Obviously none of this is usable in a portable way through the generic `error_category` API because encodings can be and often are different.

To address this, the proposal suggests using the C locale encoding (execution character set), which is already employed in most cases and aligns with underlying system APIs. Microsoft STL's implementation has a number of bugs in `std::system_category::message` ([MSSTL-3254], [MSSTL-4711]) and will likely need to change anyway. This also resolves [LWG4156].

An alternative approach could involve communicating the encoding from `error_category`. However, this introduces ABI challenges and complicates usage compared to adopting a single encoding.

§ 5. Wording

Add to "Header <system_error> synopsis" [system.error.syn]:

```
// [system.error.fmt], formatter
template<class charT> struct formatter<error_code, charT>;
```

Add a new section "Formatting" [system.error.fmt] under "Class `error_code`" [syserr.errcode]:

```
template<class charT> struct formatter<error_code, charT> {
  constexpr typename basic_format_parse_context<charT>::iterator
    parse(basic_format_parse_context<charT>& ctx);

  template<class FormatContext>
    typename FormatContext::iterator
      format(const error_code& ec, FormatContext& ctx) const;
};
```

```
constexpr typename basic_format_parse_context<charT>::iterator
  parse(basic_format_parse_context<charT>& ctx);
```

Effects: Parses the format specifier as a *error-code-format-spec* and stores the parsed specifiers in `*this`.

error-code-format-spec:
fill-and-align$_{opt}$ *width*$_{opt}$ `s`$_{opt}$

where the productions *fill-and-align* and *width* are described in [format.string].

Returns: An iterator past the end of the *error-code-format-spec*.

```
template<class FormatContext>
  typename FormatContext::iterator
    format(const error_code& ec, FormatContext& ctx) const;
```

Effects: If the `s` option is used, then:

- If the ordinary literal encoding is UTF-8, then let `msg` be `ec.message()` transcoded to UTF-8 with maximal subparts of ill-formed subsequences substituted with U+FFFD REPLACEMENT

CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion.

- Otherwise, let `msg` be `ec.message()` transcoded to an implementation-defined encoding.

Otherwise, let `msg` be `format("{}:{}", ec.category().name(), ec.value())`.

Writes `msg` into `ctx.out()`, adjusted according to the *error-code-format-spec*.

Returns: An iterator past the end of the output range.

Modify [syserr.errcat.virtuals]:

```
virtual const char* name() const noexcept = 0;
```

Returns: A string in the ordinary literal encoding naming the error category.

...

```
virtual string message(int ev) const = 0;
```

Returns: A string of multibyte characters in the executon character set that describes the error condition denoted by `ev`.

§ 6. Implementation

The proposed `formatter` for `std::error_code` has been implemented in the open-source {fmt} library ([FMT]).

§ References

§ Informative References

[FMT]
Victor Zverovich; et al. *The {fmt} library*. URL: https://github.com/fmtlib/fmt

[LWG4156]
Victor Zverovich. *`error_category` messages have unspecified encoding*. URL: https://cplusplus.github.io/LWG/issue4156

[MSSTL-3254]

Visual Studio 2022 std::system_category returns "unknown error" if system locale is not en-US. URL: https://github.com/microsoft/STL/issues/3254

[MSSTL-4711]

Sung Po-Han. *Should `std::error_code::message` respect the locale set by the user?*. URL: https://github.com/microsoft/STL/issues/4711